

tf.compat.v1.InteractiveSession Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/InteractiveSession

A TensorFlow Session for use in interactive contexts, such as a shell.

Function Signatures

```
tf.compat.v1.InteractiveSession( target='', graph=None,
config=None )
```

Code Examples

```
tf.compat.v1.InteractiveSession(
target='', graph=None, config=None
)
```

```
tf.compat.v1.InteractiveSession(
target='', graph=None, config=None
)
```

```
sess = tf.compat.v1.InteractiveSession()
a = tf.constant(5.0)
b = tf.constant(6.0)
c = a * b
# We can just use 'c.eval()' without passing 'sess'
print(c.eval())
sess.close()
```

```
sess = tf.compat.v1.InteractiveSession()
a = tf.constant(5.0)
b = tf.constant(6.0)
c = a * b
# We can just use 'c.eval()' without passing 'sess'
print(c.eval())
sess.close()
```

```
a = tf.constant(5.0)
b = tf.constant(6.0)
c = a * b
with tf.compat.v1.Session():
# We can also use 'c.eval()' here.
print(c.eval())
```

```
a = tf.constant(5.0)
b = tf.constant(6.0)
c = a * b
with tf.compat.v1.Session():
# We can also use 'c.eval()' here.
print(c.eval())
```

```
as_default()
```

```
as_default()
```

Documentation

A TensorFlow Session for use in interactive contexts, such as a shell.

Used in the notebooks

- Fitting Dirichlet Process Mixture Model Using Preconditioned Stochastic Gradient Langevin Dynamics
- Linear Mixed-Effect Regression in {TF Probability, R, Stan}

The only difference with a regular Session is that an InteractiveSession installs itself as the default session on construction.

The methods `tf.Tensor.eval`
and `tf.Operation.run`
will use that session to run ops.

This is convenient in interactive shells and IPython notebooks, as it avoids having to pass an explicit Session object to run ops.

For example:

Note that a regular session installs itself as the default session when it is created in a `with` statement. The common usage in non-interactive programs is to follow that pattern:

Attributes

`## Methods`

`as_default`

[View source](#)

Returns a context manager that makes this object the default session.

Use with the `with` keyword to specify that calls to `tf.Operation.run` or `tf.Tensor.eval` should be executed in this session.

To get the current default session, use `tf.compat.v1.get_default_session`.

Alternatively, you can use `with tf.compat.v1.Session()`: to create a session that is automatically closed on exiting the context, including when an uncaught exception is raised.

[View source](#)

Closes an `InteractiveSession`.

list_devices

[View source](#)

Lists available devices in this session.

make_callable

[View source](#)

Returns a Python callable that runs a particular step.

The returned callable will take `len(feed_list)` arguments whose types

must be compatible feed values for the respective elements of `feed_list`. For example, if element `i` of `feed_list` is a `tf.Tensor`, the `i`th argument to the returned callable must be a numpy ndarray (or something convertible to an ndarray) with matching element type and shape. See `tf.Session.run` for details of the allowable feed key and value types.

The returned callable will have the same return type as `tf.Session.run(fetches, ...)`. For example, if `fetches` is a `tf.Tensor`, the callable will return a numpy ndarray; if `fetches` is a `tf.Operation`, it will return `None`.

`partial_run`

[View source](#)

Continues the execution with more feeds and fetches. (deprecated)

This is EXPERIMENTAL and subject to change.

To use partial execution, a user first calls `partial_run_setup()` and then a sequence of `partial_run()`. `partial_run_setup` specifies the list of feeds and fetches that will be used in the subsequent `partial_run` calls.

The optional `feed_dict` argument allows the caller to override the value of tensors in the graph. See `run()` for more information.

Below is a simple example:

`partial_run_setup`

[View source](#)

Sets up a graph with feeds and fetches for partial run. (deprecated)

This is EXPERIMENTAL and subject to change.

Note that contrary to `run`, `feeds` only specifies the graph elements. The tensors will be supplied by the subsequent `partial_run` calls.

[View source](#)

Runs operations and evaluates tensors in fetches.

This method runs one "step" of TensorFlow computation, by running the necessary graph fragment to execute every `Operation` and evaluate every `Tensor` in `fetches`, substituting the values in `feed_dict` for the corresponding input values.

The `fetches` argument may be a single graph element, or an arbitrarily nested list, tuple, `namedtuple`, dict, or `OrderedDict` containing graph elements at its leaves. A graph element can be one of the following types:

- A `tf.Operation`.

The corresponding fetched value will be `None`.

- A `tf.Tensor`.

The corresponding fetched value will be a numpy ndarray containing the value of that tensor.

- A `tf.sparse.SparseTensor`.

The corresponding fetched value will be a `tf.compat.v1.SparseTensorValue` containing the value of that sparse tensor.

- A `get_tensor_handle` op. The corresponding fetched value will be a

numpy ndarray containing the handle of that tensor.

- A string which is the name of a tensor or operation in the graph.

The value returned by `run()` has the same shape as the `fetches` argument, where the leaves are replaced by the corresponding values returned by TensorFlow.

Example:

The optional `feed_dict` argument allows the caller to override the value of tensors in the graph. Each key in `feed_dict` can be one of the following types:

- If the key is a `tf.Tensor`, the

value may be a Python scalar, string, list, or numpy ndarray that can be converted to the same dtype as that tensor. Additionally, if the key is a `tf.compat.v1.placeholder`, the shape of the value will be checked for compatibility with the placeholder.

- If the key is a

`tf.sparse.SparseTensor`,
the value should be a

`tf.compat.v1.SparseTensorValue`.

- If the key is a nested tuple of Tensors or SparseTensors, the value

should be a nested tuple with the same structure that maps to their corresponding values as above.

Each value in `feed_dict` must be convertible to a numpy array of the dtype of the corresponding key.

The optional `options` argument expects a `[RunOptions]` proto. The options allow controlling the behavior of this particular step (e.g. turning tracing on).

The optional `run_metadata` argument expects a `[RunMetadata]` proto. When appropriate, the non-Tensor output of this step will be collected there. For example, when users turn on tracing in options, the profiled info will be collected into this argument and passed back.